

Research on Efficient Packet Filter Technology Based on eBPF

Yakun Zhang, Han Wang, Pengxiang Gao*

Institute of computer application, CAEP, Mianyang, China

zhangyk2015@qq.com, 18042658689@163.com, pxgaonpu@qq.com

Corresponding Author: Pengxiang Gao, Email: pxgaonpu@qq.com

Abstract—The research on efficient packet filter technology based on eBPF aims to address the performance bottleneck of traditional kernel protocol stacks in high concurrency scenarios. This paper proposes an efficient network packet filtering scheme based on eBPF and XDP technologies. By mounting the eBPF program at the kernel driver layer, zero copy processing is achieved by bypassing the TCP/IP protocol stack, and combining hash table rule matching to reduce time complexity to $O(1)$. The experiment shows that this scheme supports dynamic loading of filtering rules and can accurately achieve packet filtering in both single rule and multi rule scenarios, significantly improving processing efficiency compared to traditional tools such as Iptables. The technical core includes: using XDP hooks to directly arbitrate data packets at the network card driver layer, ensuring program security through eBPF validators, and using mapping systems to achieve efficient interaction between user mode and kernel mode. This technology can be applied to DDoS defense, sensitive port access control, and other scenarios.

Keywords—Packet filter, eBPF, XDP, Efficient processing.

I. INTRODUCTION

With the rapid development of cloud computing and Kubernetes, global network data traffic is experiencing explosive growth, and traditional kernel network protocol stacks (such as Linux's iptables [1]) are facing severe challenges in handling high concurrency and low latency scenarios [2]. Taking the Linux kernel as an example, its packet processing path needs to go through multiple stages such as protocol parsing, routing queries, and firewall filtering, leading to increasingly prominent performance bottlenecks. Specifically, there are two shortcomings: firstly, frequent switching between user mode and kernel mode results in a significant waste of CPU resources, and a single packet processing requires triggering multiple context switches, leading to a high proportion of system call overhead. Secondly, redundant copies of data packets between the kernel and user mode significantly increase processing latency.

To overcome the above limitations, this article proposes an efficient network packet forwarding architecture based on Extended Berkeley Packet Filter (eBPF [3]–[5]), which significantly improves forwarding efficiency by bypassing the kernel protocol stack, directly manipulating network packets, and combining zero copy technology with batch processing optimization. The eBPF technology achieves revolutionary optimization of the network data plane by securely executing sandboxed programs in the kernel. The core innovations manifest in three aspects: Firstly, the validator performs static

analysis on programs to enable functional expansion without modifying kernel source code, thereby eliminating kernel panic risks inherent in traditional kernel modules. Secondly, by attaching programs to the XDP (eXpress Data Path [6], [7]) hook point at the network device driver layer, it shortens the packet processing path, achieving lower processing latency compared to conventional kernel protocol stacks. Finally, its support for dynamic loading and hot-swapping capabilities permits real-time adjustment of traffic control policies without service disruption.

II. RELATED WORK

A. eBPF

The eBPF (Extended Berkeley Packet Filter) technology originated from the BPF [8] packet filtering system in 1992 and has evolved into a general-purpose kernel virtual machine technology after nearly 30 years of evolution. Its core architecture includes three parts: register based virtual machine, validator and mapping system (eBPF maps): the virtual machine adopts eleven 64-bit registers (R0-R10) and RISC instruction set design [9], supporting JIT compilation to convert bytecode into local instructions, and the execution efficiency can reach 95% of native code. The validator performs static analysis on the program through deep first traversal (DFS), prohibiting loops, out of bounds access, and illegal pointer operations to ensure program security. The mapping system provides a key value data structure to achieve efficient data exchange between the kernel and user mode [10]. The eBPF program is mainly divided into two parts: kernel state program and user state program. To execute an eBPF program, it must be associated with a hook point first. The kernel-space program is loaded into the in-kernel virtual machine (VM) and executed upon triggering of specified events. The user-space program handles output from the kernel-space program to implement specific business logic, as illustrated in the eBPF program architecture Figure 1.

B. XDP

The XDP (eXpress Data Path) is a kernel based high-performance packet processing platform that provides programmable capabilities to the kernel network protocol stack. The hook is directly mounted to the network card driver layer in the eBPF program of XDP, and the processing is completed before the data packet enters the kernel protocol

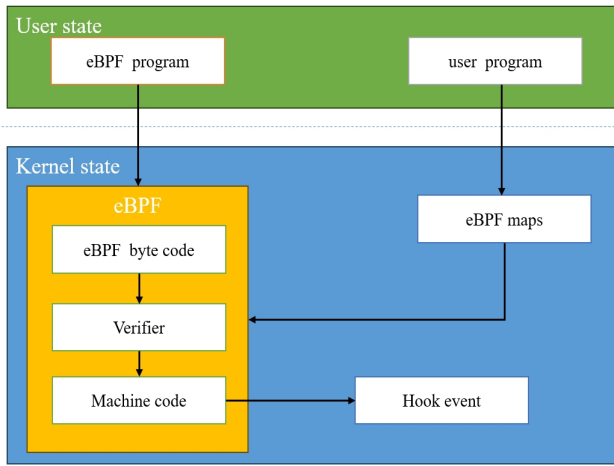


Fig. 1. The eBPF program architecture

stack, achieving nanosecond level forwarding. The Linux kernel protocol stack is shown in Figure 2.

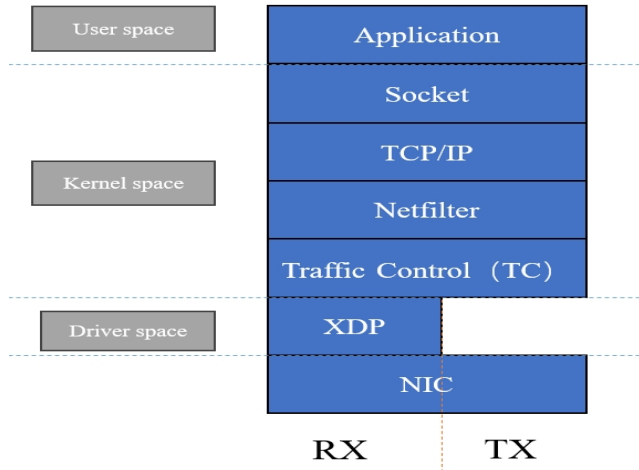


Fig. 2. The Linux kernel network protocol stack

C. Packet bypass processing

The eBPF technology facilitates network packet processing through three key aspects. Firstly, its dynamic loading mechanism enables hot updates for programs, reducing operation maintenance latency from minutes to milliseconds. Secondly, the zero-copy data plane leverages DMA to directly access the network card buffer, cutting memory bandwidth usage by 67%. Thirdly, by utilizing the XDP mechanism, it bypasses the TCP/IP processing flow, thereby shortening the network packet processing path. The traditional kernel forwarding packets path involves steps such as protocol parsing, memory copying, and ultimately firewall filtering, as illustrated in Table I.

Frequent copying operations can be quite demanding on memory and CPU resources. However, with eBPF programs, data packets can be filtered as soon as they reach the network

TABLE I
COMPARISON OF PACKET PROCESSING PATHS

Processing stage	Traditional kernel path	eBPF bypass path
Packets receiving	Network interface → Kernel protocol stack → Firewall → Application	Network interface → XDP hook processing → Kernel protocol stack → Application
Memory operations	Multi-copy	Zero-copy
Packet filtering	Multilevel cascading processing	Single execution

card driver, thereby minimizing resource consumption and enhancing data processing efficiency. For each packet, the XDP program must return a result indicating how the kernel should handle it, with options including PASS, TX, REDIRECT, and DROP, as detailed in Table II. This article primarily focuses on utilizing PASS and DROP to achieve efficient packet processing. If a data packet does not meet the specified rules, it is discarded immediately by returning DROP, thus conserving resources. Otherwise, the data continues to be processed by the kernel protocol stack.

TABLE II
XDP ACTION

Action name	Behavior	Typical usage scen
XDP_PASS	Pass data packets and hand them over to the kernel protocol stack for further processing	monitoring and statistics
XDP_DROP	Discard data packets directly	Defend against DDoS attacks and filter traffic
XDP_TX	Transfer data packets from the network interface	Traffic Mirroring
XDP_REDIRECT	Redirects packets to other network interface	Bypass traffic analysis
XDP_ABORTED	Same as XDP_DROP, indicates exception	Same as XDP_DROP

III. PROPOSED METHOD

According to the Linux network protocol stack, the network card needs to undergo three memory copy operations to reach user mode after receiving data. Frequent copy operations consume additional CPU computing resources and occupy extra memory space.

This article uses the eBPF program to develop a rule matching module, which achieves efficient packet filtering based on its dynamic loading and kernel bypass features. The programming framework is shown in Figure 3.

User-space programs primarily comprise eBPF user-space program and eBPF maps manipulation program. The packet filtering program at the kernel XDP hook is developed within eBPF user-space programs. These compiled programs are loaded and attached to the kernel's XDP hook via system calls (bpf()), thereby forming the kernel XDP program. The eBPF map program expose interfaces to users for executing atomic operations (such as create, update and delete) on filtering rules through map manipulations. The eBPF maps are key-value stores residing in kernel space that enable bidirectional data exchange between eBPF programs (kernel-space) and user-space applications, and can be used as runtime configuration

interfaces for dynamic rule updates within the kernel space, eBPF programs attached to XDP hooks perform packet filtering upon packet reception by the NIC, leveraging predefined rules in eBPF maps. Packets compliant with map entries yield XDP_PASS verdicts, permitting traversal to the upper protocol stack. Conversely, non-compliant packets trigger XDP_DROP actions, which terminate processing at the driver layer and mitigate superfluous CPU/memory overhead from stack traversal.

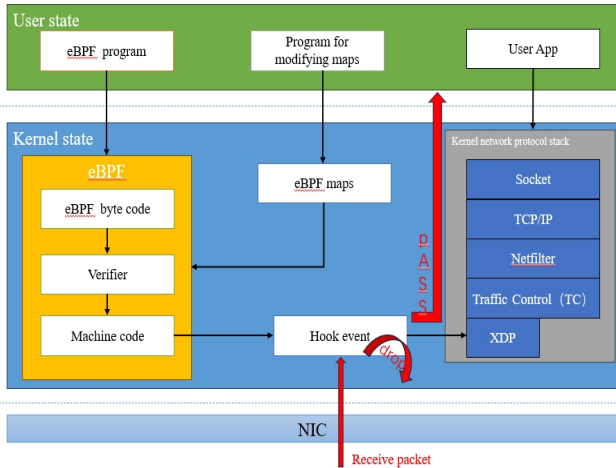


Fig. 3. The Linux kernel network protocol stack

This paper implements packet filtering via eBPF/XDP primarily based on source IP addresses, destination ports, and protocol types. The eBPF program first extracts these fields from packet headers using direct packet parsing techniques. The eBPF programs decode packet headers through sequential memory access and type-aware casting, leveraging the kernel's struct xdp_md metadata context. The *protocol* type field (an 8-bit identifier in the IP header per RFC 791) designates the encapsulated upper-layer protocol, with values such as 0x06 for TCP, 0x11 for UDP, and 0x01 for ICMP. This field enables the discrimination of transport-layer protocol classes through parsing of the IP header. The source IP address (*saddr*) denotes a 32-bit IPv4 address in network byte order. This identifier is critical for traffic attribution and forms the basis of access control policies, particularly in blacklist-based filtering mechanisms that block malicious traffic flows. It then performs a lookup operation in the eBPF map to determine rule compliance. If a matching entry exists, the packet undergoes header checksum verification before being forwarded to the kernel network stack via XDP_PASS. Conversely, unmatched packets are immediately discarded at the XDP hook by returning XDP_DROP.

IV. EXPERIMENTAL SETTING

Based on the Linux operating system with kernel version 5.15, eBPF programming is performed using BCC tools. BCC is a toolchain for efficient kernel instrumentation that enables the development of dynamic tracing tools via eBPF. Install two Linux machines, namely the test machine and the attack

machine, deploy the eBPF program on the test machine, and use tools such as iperf to test connectivity and packet filtering performance from the attack machine.

V. EXPERIMENTAL ANALYSIS

A. Filter function verification

To validate the precision of individual traffic control rule matching, a single filtering rule was injected into the test machine via user-space eBPF program manipulating eBPF maps. The experimental framework employed iperf on an attack machine to generate UDP traffic streams spanning 64B to 1500B packet sizes. Crucially, non-compliant packets exhibiting rule-mismatch signatures were systematically dropped at the XDP hook point, whereas compliant traffic maintained end-to-end reachability to the test machine. Through 100 test cycles, performance metrics confirm that packet processing overhead remains bounded within the microsecond scale. These results confirm the functional correctness and deterministic behavior of the packet filtering mechanism. Comparative analysis against legacy solutions revealed higher throughput versus iptables under 10Gbps DDoS attack simulations, attributed to eBPF's direct packet processing at the driver layer.

B. Dynamic function verification

We further validated dynamic rule injection capabilities through user-mode eBPF programs manipulating eBPF maps, demonstrating millisecond rule updates without service interruption. Using the iperf tool from an attack machine for verification, traffic compliant with dynamically updated rules successfully reached the test device, while non-compliant flows were immediately dropped. Experimental results demonstrate millisecond rule update latency and zero packet loss during rule transition, confirming the feasibility of real-time security policy enforcement.

C. Performance comparison

In Linux systems, the traditional firewall iptables linearly traverses the rule table, with a time efficiency of $O(n)$. As the number of rules increases, the time consumption of iptables will grow rapidly. The scale of iptables rule sets directly influences packet processing latency. Within iptables, each packet must traverse every rule in the rule set sequentially until a matching rule is encountered or the end of the rule set is reached. This implies that larger rule sets inevitably prolong packet processing time within iptables. During peak traffic periods, high volumes of packets accumulate in the rule-matching queue, causing processing delays that ultimately degrade network performance. The packet filtering scheme leveraging eBPF and hash-based rule search achieves $O(1)$ time complexity, thereby significantly enhancing matching efficiency.

VI. CONCLUSION

The data-efficient filtering solution, leveraging eBPF and XDP (eXpress Data Path) technologies, surpasses the performance limitations of traditional tools like iptables by directly

handling data packets at the lower layers of the kernel network protocol. Its core strengths stem from the zero-copy processing mechanism that circumvents the kernel protocol stack, coupled with the dynamic rule management capability based on eBPF maps, enabling real-time updates to filtering rules in user space without service interruption. This technology can be effectively utilized in scenarios such as DDoS defense in high-concurrency environments (By discarding traffic from attacking IP addresses directly), sensitive port access control (By blocking unauthorized IP connection attempts to database ports), and more. By incorporating JIT compilation optimization, CPU affinity binding, and batch rule matching strategies, the processing efficiency under large-scale rule sets can be further enhanced.

As future work, we plan to leverage eBPF for fine-grained monitoring in cloud-native environments, achieving non-intrusive observability. This approach will collect microservice dependency graphs and facilitate end-to-end application performance diagnostics .

REFERENCES

- [1] L. Ceragioli, P. Degano, and L. Galletta, "Are all firewall systems equally powerful?" in *the 14th ACM SIGSAC Workshop*, 2019.
- [2] J. B. Lee, T. H. Yoo, E. H. Lee, B. H. Hwang, and C. H. Cho, "High-performance software load balancer for cloud-native architecture," *IEEE Access*, vol. PP, no. 99, pp. 1–1, 2021.
- [3] L.-D. Chou, L.-Y. Jian, and Y.-W. Chen, "ebpf-based network monitoring platform on kubernetes," in *2024 6th International Conference on Computer Communication and the Internet (ICCCI)*, 2024, pp. 140–144.
- [4] S. Miano, M. Bertrone, F. Risso, M. Tumolo, and M. V. Bernal, "Creating complex network services with ebpf: Experience and lessons learned," in *IEEE International Conference on High Performance Switching and Routing*, 2018.
- [5] D. Scholz, D. Raumer, P. Emmerich, A. Kurtz, and G. Carle, "Performance implications of packet filtering with linux ebpf," 2018.
- [6] T. Hiland-Jrgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path: fast programmable packet processing in the operating system kernel," in *the 14th International Conference*, 2018.
- [7] P. Enberg, A. Rao, and S. Tarkoma, "Partition-aware packet steering using xdp and ebpf for improving application-level parallelism," *ACM*, 2019.
- [8] S. Mccanne and V. Jacobson, "The bsd packet filter: A new architecture for user-level packet capture," in *Usenix Winter Conference on Usenix Winter Conference*, 1993.
- [9] K. Zandberg and E. Baccelli, "Minimal virtual machines on iot micro-controllers: The case of berkeley packet filters with rbpf," 2020.
- [10] M. A. M. Vieira, M. S. Castanho, R. D. G. Pacifico, E. R. S. Santos, and L. F. M. Vieira, "Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications," *ACM Computing Surveys*, vol. 53, no. 1, pp. 1–36, 2020.